

**MODUL PRAKTIKUM MIKROPROSESOR & ANTARMUKA
ROBOTIC & EMBEDDED SYSTEM LABORATORY**



**Tugas Pendahuluan Modul 2
CMSIS Programming**

Nama : Ariski Myardi
NIM : 1103220019
Shift/Hari : 3/Rabu
Kelompok : 30

NIM Ganjil

1. Jelaskan peran CMSIS dalam pengembangan perangkat lunak berbasis mikrokontroler ARM Cortex-M!
2. Bagaimana cara mengatur sebuah pin GPIO menjadi mode output pada STM32 menggunakan CMSIS?
3. Apa itu bit-banding dan bagaimana hal ini dapat meningkatkan efisiensi pemrograman pada mikrokontroler ARM Cortex-M?
4. Deskripsikan cara kerja seven-segment display dan bagaimana metode multiplexing dapat digunakan untuk mengontrol lebih dari satu seven-segment dengan jumlah pin GPIO yang terbatas!
5. Jelaskan peran CMSIS-CORE dalam pemrograman mikrokontroler berbasis ARM Cortex M dan bagaimana CMSIS-CORE memberikan akses ke register hardware?

Jawaban:

1. CMSIS (Common Microcontroller Software Interface Standard) merupakan suatu standar antarmuka perangkat lunak yang sangat penting dalam pengembangan mikrokontroler, terutama berbasis ARM Cortex-M.
2. Cara mengatur pin GPIO
 - Setiap pin pada mikrokontroler dapat dikonfigurasi sebagai input atau output
 - Konfigurasi pin ini dilakukan melalui MODER (GPIO Mode Register). Dalam CMSIS, register ini dapat diakses dengan GPIOx->MODER, di mana x adalah port GPIO (misalnya, A, B, C)
 - Untuk menulis nilai digital ke pin, digunakan register ODR (Output Data Register). Setiap bit pada register ini mewakili satu pin.
3. Bit-banding pada mikrokontroler ARM Cortex-M adalah teknik yang memungkinkan manipulasi bit individual dalam memori secara efisien. Teknik ini memetakan setiap bit dari area memori tertentu (SRAM atau Peripheral) ke alamat alias dalam bit-band region, sehingga memungkinkan pengubahan satu bit tanpa memengaruhi bit lainnya dalam byte atau word yang sama.
4. Seven-segment display menampilkan angka dengan menyalakan kombinasi dari tujuh segmen

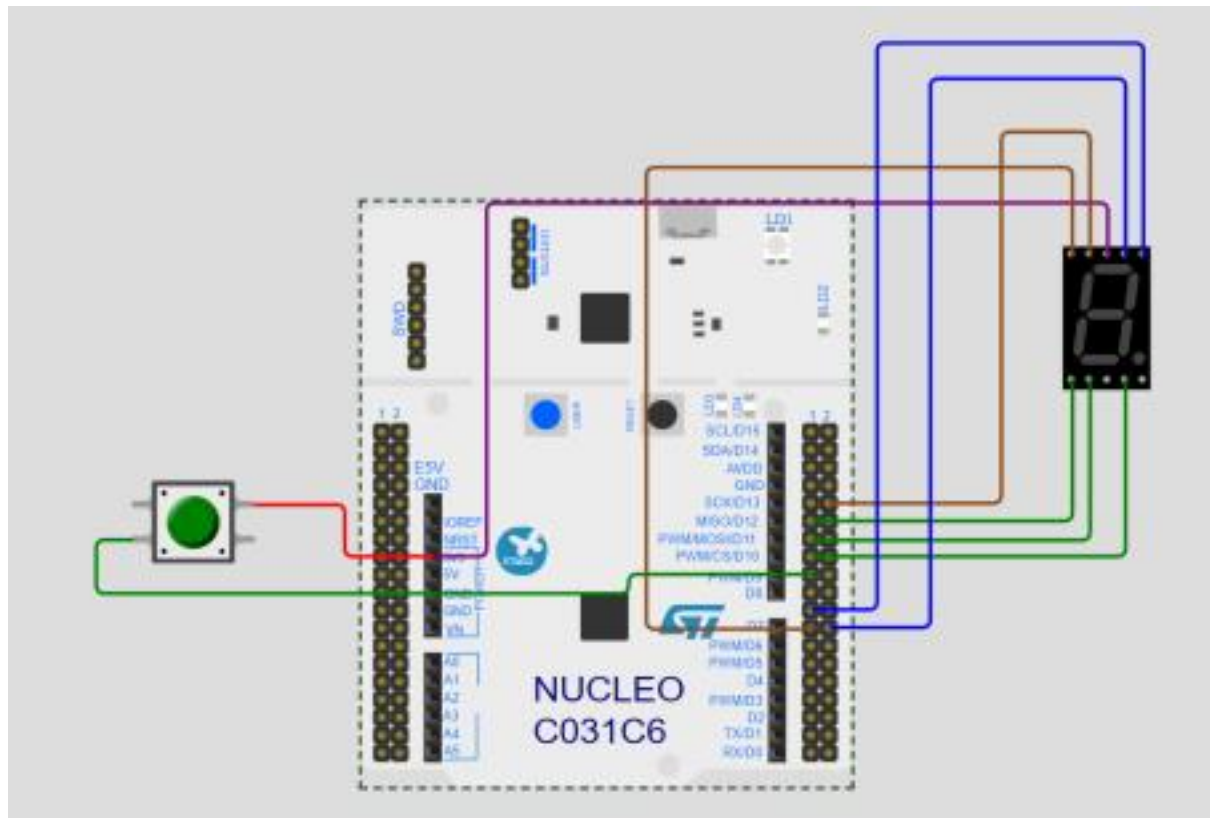
LED (a-g). Untuk mengontrol beberapa display dengan jumlah pin GPIO terbatas, digunakan metode multiplexing, yaitu menyalakan satu display pada satu waktu secara bergantian dengan sangat cepat. Segmen-segmen dari semua display dihubungkan ke pin GPIO yang sama, sementara setiap display memiliki pin kontrol tersendiri. Dengan memindai tiap display secara cepat, tampilan terlihat menyala bersamaan oleh mata manusia, meskipun sebenarnya hanya satu display yang aktif pada satu waktu.

5. CMSIS-CORE adalah antarmuka standar untuk pemrograman mikrokontroler ARM Cortex-M yang menyediakan akses langsung ke register hardware dan sistem interrupt. Ini memungkinkan pengembang mengontrol fungsi-fungsi perangkat keras prosesor seperti register CPU, NVIC (Nested Vectored Interrupt Controller), dan pengaturan clock dengan cara yang sederhana dan konsisten.

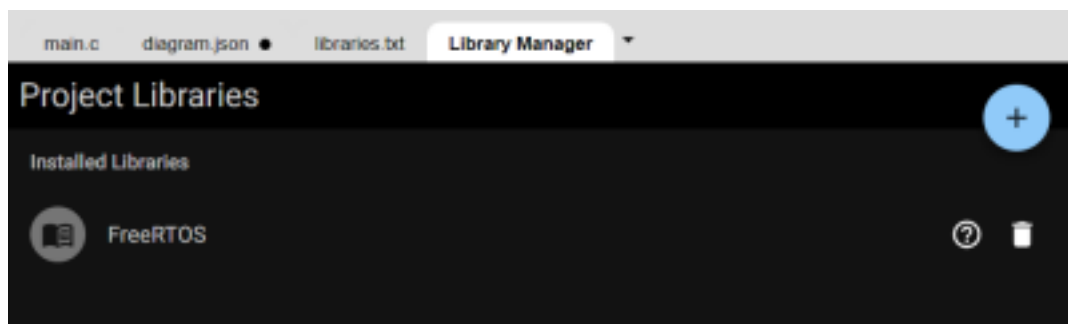


Soal Praktik

Buatlah rangkaian simulasi untuk mengontrol tampilan seven-segment display dari 0 hingga 9 menggunakan push button pada board Nucleo 64 di Wokwi.



Tambahkan library FreeRTOS di dalam Library Manager.



Gunakan file main.c untuk menjalankan kode dibawah ini.

```
#include "stm32c0xx_hal.h"

#define A GPIO_PIN_8
#define B GPIO_PIN_9
#define C GPIO_PIN_7
#define D GPIO_PIN_6
```

```
#define E GPIO_PIN_5
#define F GPIO_PIN_3
#define G GPIO_PIN_15
```

```
int tab[10][7] = {
// a b c d e f g
{0, 0, 0, 0, 0, 0, 1}, // 0
{1, 0, 0, 1, 1, 1, 1}, // 1
{0, 0, 1, 0, 0, 1, 0}, // 2
{0, 0, 0, 0, 1, 1, 0}, // 3
{1, 0, 0, 1, 1, 0, 0}, // 4
{0, 1, 0, 0, 1, 0, 0}, // 5
{0, 1, 0, 0, 0, 0, 0}, // 6
{0, 0, 0, 1, 1, 1, 1}, // 7
{0, 0, 0, 0, 0, 0, 0}, // 8
{0, 0, 0, 0, 1, 0, 0} // 9
};
```

```
int i = 0;
```

```
void SystemClock_Config(void);
```

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    GPIO_InitTypeDef GPIO_InitStruct;
```

```
/* GPIO Ports Clock Enable */
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
```

```
/* Configure GPIO pins */
GPIO_InitStruct.Pin = A | B | C | D | E | F | G;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

```
/* Configure the button pin */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

```
while (1)
{
    // Check if the button is pressed
    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_SET)
```

**MODUL PRAKTIKUM MIKROPROSESOR &
ANTARMUKA ROBOTIC & EMBEDDED SYSTEM
LABORATORY**



```

{

HAL_GPIO_WritePin(GPIOA, A, tab[i][0]);
HAL_GPIO_WritePin(GPIOA, B, tab[i][1]);
HAL_GPIO_WritePin(GPIOA, C, tab[i][2]);
HAL_GPIO_WritePin(GPIOA, D, tab[i][3]);
HAL_GPIO_WritePin(GPIOA, E, tab[i][4]);
HAL_GPIO_WritePin(GPIOA, F, tab[i][5]);
HAL_GPIO_WritePin(GPIOA, G, tab[i][6]);

i = (i + 1) % 10;
HAL_Delay(200);
}

}
}

void SystemClock_Config(void)
{
RCC_OscInitTypeDef RCC_OscInitStruct;
RCC_ClkInitTypeDef RCC_ClkInitStruct;

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
//RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE; // No PLL for HSI

if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
Error_Handler();
}

RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) !=
HAL_OK)
{
Error_Handler();
}
}

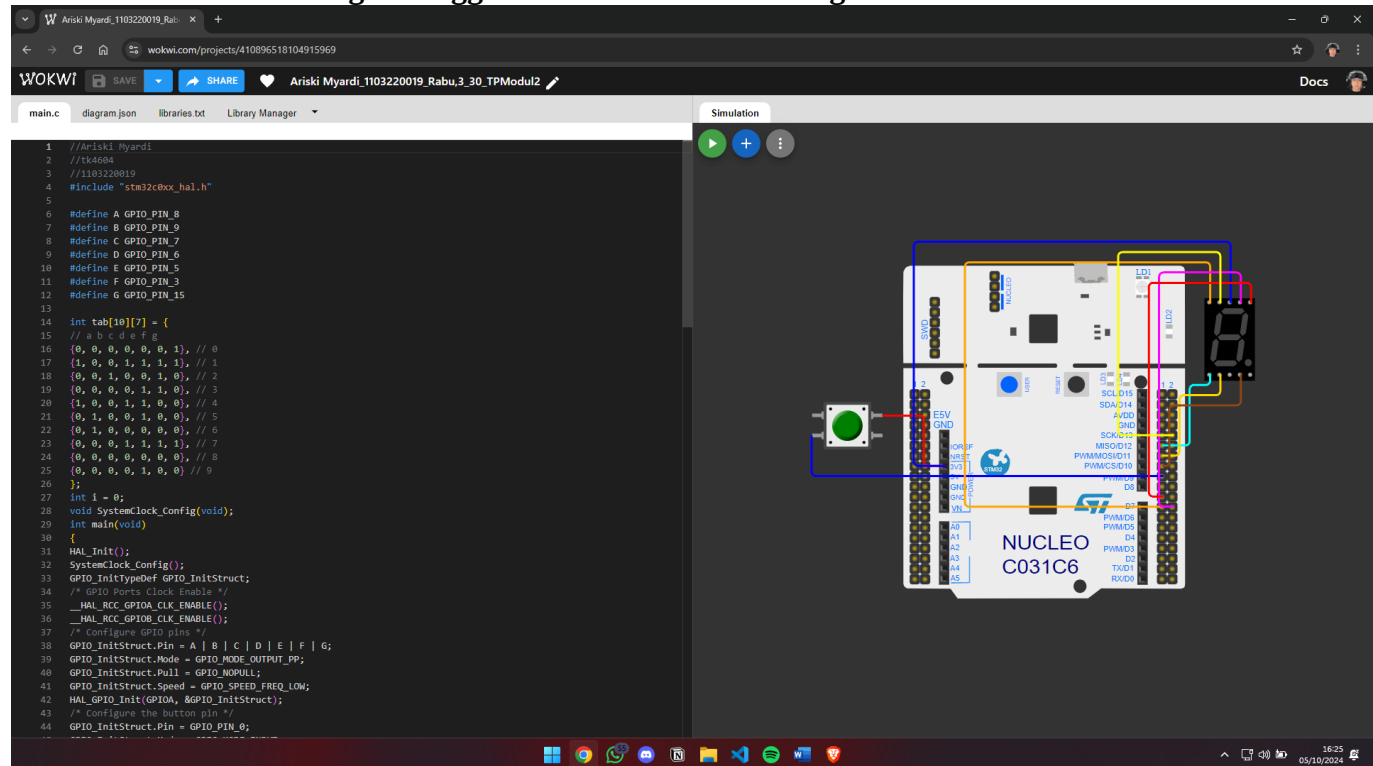
void Error_Handler(void)
{

```



```
// Handle errors appropriately, e.g., blink an error LED or report the error through a
debug interface.
while (1)
{
// Infinite loop to keep the microcontroller in an error state
}
}
```

***Berikan screenshot dengan tanggal dan waktu disertai dengan link wokwi!**



<https://wokwi.com/projects/410896518104915969>